DOCUMENTATION · PERFORMANCE

Performance, v3: an OLTP benchmark on PostgreSQL

The latest version is at anantharajuc.github.io/QueryAPIGate-website/Guide.dc.html?d=PERFORMANCE_V3. This copy was made on 4 November 2026.

> **0.43's figures are in Performance, v4,** which reruns this benchmark against 0.42. This page stays the full description of the method, and the reference for the blocks v4 didn't rerun.

The short answer. One gunicorn worker (as in the Docker image) in front of PostgreSQL, serving a saved query by primary key to 32 concurrent callers:

	V0.41.0	`MAIN` (UNRELEASED 0.42)	`MAIN` WITH THE METADATA CACHE
Requests a second	682	743	875
One caller at a time, p50	1.43 ms	1.38 ms	1.05 ms
With a governed key (masking and a table allow-list)	171	693	802

- **The governance features cost about 7% now.** A key with masking and a table allow-list is about 4× faster than in v0.41.0.
- **One process serves 730–875 requests a second.** p99 stays at about 4 ms up to 75% of that, then climbs.
- **More processes add throughput.** Eight workers reached 2,165–2,727 requests a second on four physical cores.
- **PostgreSQL is never the limit here.** At most it used 2 of the 4 hardware threads it had.
- **The remaining cost of a call** is about 1 ms of Python (logging alone takes a fifth of it), plus its round trips to PostgreSQL. A call on `main` makes 7: 3 to QueryAPIGate's own store and 4 to the data database. With the metadata cache, it makes 4. Over a real network, those round trips dominate.

For MySQL - one server, a database per microservice, per-user queries joining across them - see Performance on MySQL.

How to read the numbers. Every comparison is the median of 5 interleaved rounds, with its min–max range. A difference counts as real only when the two ranges don't overlap. The cold-path test and the profile are single long runs, marked where they appear. This page replaces Performance, v2: What changed from v2 lists what its review found and what was done about each point.

Contents

- Method
- Results
- Where the time goes
- Sizing
- What to do with this
- What changed from v2
- Limits
- Run it yourself

Method

Two sessions

The results come from two sessions on the same machine, with the same harness and settings:

SESSION	WHEN	BLOCKS	`/HEALTH` FLOOR
A	2026-10-23, 20:59–23:24	pgbench, throughput, single-call latency, cold path, access log, profile	1,934–2,296 req/s
B	2026-10-24, 08:45–11:08	open loop, scale-out, network, mixed and hot rows, threads, round trips	1,959–2,258 req/s

Session A also ran the session-B blocks with 3 rounds each, for v2. A review of v2 pointed out that 3 rounds is too few for the comparison rule below, so session B reran those blocks with 5. Only session B's numbers are used for them here. The two sessions agree where they overlap: one worker at 32 callers served 743 lookups a second on `main` in session A, and 730 in session B.

The machine, and how it was kept steady

One laptop: an Intel i7-12700H with 6 performance cores (CPUs 0–11, two threads each) and 8 efficiency cores (CPUs 12–19), 32 GB of memory, Ubuntu with kernel 6.8.

- **On AC power,** with the power profile and the CPU's energy preference both set to `performance`. Turbo stays on, because turning it off needs root. Its effect is measured below.
- **Each part on its own cores**, so they don't take CPU from each other:

PART	CPUS	HOW
QueryAPIGate (gunicorn)	0–7: four performance cores, both threads of each	`taskset -c 0-7`
PostgreSQL	8–11: two performance cores	`docker run --cpuset-cpus 8-11`
Load generator (oha) and pgbench	12–19: the efficiency cores	`docker run --cpuset-cpus 12-19`

- **No proxy in the path.** The gateway reaches PostgreSQL at the container's own address, not through Docker's published port, whose userland proxy is another unpinned process copying every byte.
- **A floor before and after every block:** 10 s of `/health` on one worker, the same work every time, so a machine that changed speed shows up. Both sessions sagged about 10% in their last hour, after about 90 minutes of load. That's why every table compares only within its own block, where the builds take turns.
- **The load generator was never the bottleneck.** It used at most 0.79 of its 8 cores, at 2,700 requests a second.

Turbo, measured

The busiest gateway core's clock was sampled every 2 s of every run. It ranged from 2.4 to 4.4 GHz overall, but that range is between blocks: with 8 busy workers, the cores run at a lower all-core turbo. Within a cell - the same build, call and setting - the clock varied by only ±1% (5th to 95th percentile, over 184 runs).

Turbo still explains the outliers. Within cells, a run's clock and its throughput correlate ($r = 0.59$), and the rare fast rounds are clock spikes. In session A, a v0.41.0 round ran at 4.09 GHz against its cell's 3.65, and served 11% more. That's the reason for 5 rounds and the range rule: one fast round widens a range rather than moving a median.

When a difference counts

Builds take turns in an order rotated each round (A B C, then B C A, then C A B), so none always runs first, warmest or coolest. A difference counts as real only when the two cells' ranges don't overlap.

How often does that rule call a difference real when there isn't one? If two builds were really the same, every ordering of their 2n results would be equally likely, and the ranges would be clear in 2 of the C(2n, n) orderings. For 5 rounds that's 2 in 252, under 1%. For 3 rounds it's 2 in 20, 10%, which is why v2's 3-round blocks were rerun. This page makes about 40 such comparisons, so expect perhaps one of the narrow ones to be luck. The large ones aren't.

The builds

LABEL	WHAT	COMMIT
v0.41.0	the 0.41.0 release	`f61d4ed`
main	`main` after the performance work in #88, unreleased - what 0.42 will ship	`c9b15c2` in session A, `99db30b` in session B: the same `queryapigate/` code; only docs changed between them
main+cache	the same, with `QUERYAPIGATE_METADATA_CACHE_TTL=5`	as main

Each is exported from git (`git archive`) and run as the Docker image runs it: gunicorn with the `gthread` worker, 8 threads, the access log on, everything else at its default (so v0.41.0 has a pool of 5, `main` a pool of 8).

Software: Python 3.12.3, gunicorn 26.2.0, Flask 3.1.1, psycopg2 2.9.10, sqlglot 25.34.1, PostgreSQL 16.15 (`postgres:16`), oha 1.16.0 pinned by digest (`ghcr.io/hatoo/oha@sha256:3ec3dbf549ea197793482d47a6324797411406bbf438c2fe8b91f244ec641a2f`).

The database

pgbench's TPC-B-like schema at scale 100 (`pgbench -i -s 100`), 1.5 GB, fully in memory (`shared_buffers=2GB`, `max_connections=300`, `synchronous_commit=on`):

TABLE	ROWS	COLUMNS	KEY
`pgbench_accounts`	10,000,000	`aid integer`, `bid integer`, `abalance integer`, `filler character(84)`	primary key `aid`
`pgbench_branches`	100	`bid integer`, `bbalance integer`, `filler character(88)`	primary key `bid`
`pgbench_tellers`	1,000	`tid integer`, `bid integer`, `tbalance integer`, `filler character(84)`	primary key `tid`
`pgbench_history`	0	`tid`, `bid`, `aid`, `delta integer`, `mtime timestamp`, `filler character(22)`	none

QueryAPIGate's own store - keys, saved queries, connections, run history - is a second database, `qagstore`, in the same PostgreSQL, as `QUERYAPIGATE_DATABASE_URL`.

The calls and the keys

Three saved queries on one connection, `oltp`, called with a random id on every request:

SAVED QUERY	SQL	CALLED AS
`account` - a **lookup**	`SELECT aid, bid, abalance FROM pgbench_accounts WHERE aid = :aid`	`/q/account?aid=` 1 to 9,999,999
`account_page` - a **100-row page**	`SELECT a.aid, a.abalance, b.bid, b.bbalance FROM pgbench_accounts a JOIN pgbench_branches b ON b.bid = a.bid WHERE a.aid BETWEEN :start AND :start + 99 ORDER BY a.aid`	`/q/account_page?start=` 1 to 999,999 `&page_size=100`
`deposit` - a **write**	`UPDATE pgbench_accounts SET abalance = abalance + :delta WHERE aid = :aid`	`/q/deposit?aid=` 1 to 9,999,999 `&delta=` 1 to 9

KEY	GRANTS
plain	`{"queries": ["account", "account_page"], "connections": []}`
governed	the same, plus `"allowed_tables": ["pgbench_accounts", "pgbench_branches"]` and `"mask": [{"column": "*.abalance", "action": "hash"}]`
writer	`{"queries": ["deposit"], "connections": [], "allow_writes": true, "allowed_write_ops": ["update"]}`, on a server with `QUERYAPIGATE_ALLOW_WRITES=1`

`benchmarks/oltp.py` creates all of them through the Management API. Across the two sessions, every one of the 12 million requests that completed returned 200. Requests still in flight when a run's time was up aren't counted either way.

The load

- **Closed loop**, for throughput: `oha -c N` keeps N callers, each sending its next request when the last one answers. 30 s per run, after a 5 s warm-up at 8 callers that also fills the pool and the governance caches.
- **Open loop**, for latency against load: `oha -q R --latency-correction` sends R requests a second whatever happens, and measures each request from when it should have been sent. That avoids coordinated omission, where a closed loop under-reports latency at saturation because a slow answer delays the next request. The rates are 25% to 110% of capacity, 20 s each, with 128 connections available. Capacity is measured first in the same block: 5 closed-loop rounds at 32 callers.

What is recorded for each run

- **Throughput and latency:** requests a second, the success rate and every status code; p50, p90, p99 and p99.9, from oha's own percentiles.
- **CPU, in cores**, for each part:
  - the gateway's, from `/proc` (master and workers);
  - PostgreSQL's, from its container's cgroup (`cpu.stat`);
  - the load generator's cores, from `/proc/stat`.
- **Transactions per request**, on the store and on the data database, from `pg_stat_database`. These are exact for 30 s runs. For exact *statement* counts, `log_statement = 'all'` was turned on for a few hundred requests (see Round trips).
- **The machine's state:** the busiest gateway core's clock, and the CPU package temperature (65-100 °C).

The data is in `benchmarks/results/oltp-postgres-2026-10-23/` (session A, with the cold-path per-second latency and the py-spy profiles) and `benchmarks/results/oltp-postgres-2026-10-24/` (session B). Each has `results.jsonl`, one line per run, and `results.md`, every table, generated by `benchmarks/oltp_report.py`.

Results

The database on its own

pgbench from the load generator's cores, 5 rounds of 10 s (session A):

WORKLOAD	CLIENTS	TRANSACTIONS A SECOND	AVERAGE LATENCY
Primary-key read (`pgbench -S`)	1	38,289 (37,465–39,586)	0.026 ms
Primary-key read (`pgbench -S`)	32	112,321 (96,140–123,670)	0.285 ms
TPC-B: three updates, a select, an insert	32	12,970 (12,617–15,293)	2.5 ms

A lookup costs PostgreSQL 0.03 ms on two cores. Everything a call through QueryAPIGate takes beyond that is the gateway and its round trips.

One process, 32 callers

Session A. **Cached** is `main` with the metadata cache on.

CALL		V0.41.0	MAIN	CACHED
Lookup, plain key	req/s	682 (679–686)	743 (736–746)	875 (869–876)
	p99	55.8 ms	54.1 ms	45.1 ms
Lookup, governed key	req/s	171 (171–172)	693 (691–694)	802 (800–806)
	p99	223 ms	57.1 ms	49.9 ms
100-row page	req/s	596 (595–598)	639 (638–642)	738 (737–740)
	p99	64.8 ms	61.4 ms	52.7 ms
Write	req/s	682 (679–683)	738 (733–743)	875 (872–881)
	p99	56.9 ms	53.9 ms	45.9 ms

All the ranges are clear of each other:

- **main against v0.41.0:** +7 to 9% for plain keys, from reading the store less (below); +304% for a governed key, from the masking and table-check caches.
- **Cached against main:** +16 to 19%, on every call.
- **The gateway used 1.2 cores for one worker:** one for Python, the rest for the threads' I/O. PostgreSQL used 0.2–0.4.

One caller at a time

The latency of a single call (session A):

CALL		V0.41.0	MAIN	CACHED
Lookup, plain key	p50	1.43 ms (1.43–1.44)	1.38 ms (1.35–1.42)	1.05 ms (1.04–1.06)
	p99 / p99.9	2.1 / 3.0 ms	2.2 / 3.1 ms	1.7 / 2.4 ms
Lookup, governed key	p50	6.00 ms (5.79–6.06)	1.48 ms (1.46–1.51)	1.20 ms (1.16–1.20)
	p99 / p99.9	7.6 / 13.6 ms	2.4 / 3.3 ms	1.9 / 2.5 ms

- **For one plain caller, main and v0.41.0 overlap:** three fewer store reads don't show at this speed. The cache's −24% does.
- **A governed call** went from 6 ms to 1.5.

Latency against offered load

The lookup, open loop and latency-corrected, 20 s per rate (session B). Capacity, measured first, was 730 (720–797) requests a second for main and 862 (840–872) cached. The cells are p50 / p99 / p99.9:

OFFERED	MAIN	CACHED
25%	2.0 / 3.0 / 11.0 ms	1.5 / 2.3 / 10.5 ms
50%	1.6 / 2.8 / 8.4 ms	1.1 / 2.1 / 9.8 ms
75%	1.5 / 3.8 (3.2–4.1) / 13.5 ms	1.2 / 3.4 (3.3–4.6) / 15.0 ms
90%	1.7 / 11.1 (7.6–12.9) / 21.7 ms	1.2 / 8.7 (6.3–11.6) / 19.0 ms
100%	1.9 / 22.7 (18.3–67.2) / 27.8 ms	1.6 / 17.2 (15.2–23.6) / 21.8 ms
110%	664 / 1,584 / 1,607 ms; 744 of 802 served	305 / 1,276 / 1,296 ms; 893 of 948 served

- **p50 stays flat** right up to capacity. **p99 holds at about 4 ms to 75%,** then climbs: about 3× at 90%, 6× at 100%.
- **Past capacity, a queue builds.** Latency reaches seconds within the 20 s run. Requests that were still queued when the time was up are neither served nor failed in these counts.
- **Real capacity is a little higher than measured.** At 110%, more was served than the closed-loop capacity: 744 against 730, and 893 against 862. The open loop's 128 connections get 2–4% more out of a worker than 32 callers do. So each row's real load sits a few percent below its label, and the 75% planning figure below errs on the safe side.
- **The lightest loads show a higher p99.9 than 50%:** at a trickle, an idle core clocks down between requests.

Scale-out

Workers in one gunicorn (each a process, as each container would be), 64 callers (session B):

WORKERS	LOOKUPS REQ/S: MAIN	LOOKUPS: CACHED	WRITES: MAIN	WRITES: CACHED
1	734 (733–739)	869 (866–871)	732 (729–733)	863 (844–866)
2	1,196 (1,193–1,202)	1,441 (1,431–1,445)	1,188 (1,185–1,190)	1,430 (1,428–1,434)
4	1,716 (1,709–1,722)	2,112 (2,104–2,124)	1,706 (1,701–1,710)	2,087 (2,078–2,094)
8	2,215 (2,127–2,264)	2,727 (2,638–2,801)	2,165 (2,096–2,229)	2,651 (2,552–2,683)

WORKERS	P99, LOOKUPS: MAIN / CACHED	GATEWAY CORES	LOOKUPS/S PER GATEWAY CORE: MAIN / CACHED	POSTGRESQL CORES: MAIN / CACHED
1	103 / 88 ms	1.2	612 / 736	0.30 / 0.17
2	74 / 59 ms	2.3-2.4	501 / 622	0.56 / 0.31
4	66 / 69 ms	4.6-4.7	367 / 463	1.04 / 0.56
8	139 / 92 ms	7.2	307 / 377	1.65 / 0.92

- **The cache's lead is clear at every size:** +18% to +23%.
- **Throughput grows with processes, but not linearly here,** and the table shows why. Requests a second per gateway core fall from 612 at 1 worker to 307 at 8. All 8 workers share CPUs 0-7, which are only four physical cores. From 4 workers on, the workers share cores' second hyperthreads, and a hyperthread is worth far less than a core. Containers on their own cores, or their own hosts, would scale better than this. This run can't show by how much.
- **The cache halves PostgreSQL's work.** At 8 workers it used 1.65 cores without the cache and 0.92 with it. Most of PostgreSQL's load was QueryAPIGate's own store, not the queries.
- **p99 at 8 workers rises,** and its range widens (84–184 ms without the cache): 8 processes and their threads contend for 8 hardware threads.

Threads per worker

One worker, the lookup, 32 callers (session B):

THREADS	REQ/S	P99	GATEWAY CORES
1	788 (782-792)	51.7 ms	0.87
2	843 (842-851)	47.5 ms	1.12
4	777 (772-778)	51.0 ms	1.19
8	735 (734-737)	54.5 ms	1.20

All four ranges are clear of each other.

- **Two threads are best on this workload:** one runs Python while the other waits on the database.
- **Beyond two, throughput falls.** Eight threads, the Docker image's setting, are 13% slower than two, and 7% slower than a single thread. They compete for Python's global interpreter lock and, the profile shows, for the logging module's lock.
- **That applies to fast queries only.** Threads are for calls that wait: slow queries, `?stream=true` exports, and `/events` connections that stay open. See Sizing.

The cold path: every 60 seconds

One 150 s run of the lookup at 8 callers per key (session A, a single run each), with each request's duration from gunicorn's access log (`%(D)s`):

KEY	REQUESTS	P50	P99	P99.9	P99.99	SLOWEST
governed	106,741	9.25 ms	14.66 ms	19.2 ms	73.7 ms	146.9 ms
plain	115,314	8.58 ms	13.57 ms	16.9 ms	20.9 ms	26.0 ms

The log starts with the server, so the 5 s warm-up comes first. In a typical second the slowest request took 17 ms; three seconds stand out:

- **t=1 s, 147 ms:** the very first governed calls, which read the connection's schema and trace the statement for masking for the first time.
- **t=61 s and t=121 s, 76 and 74 ms:** the same work again, 60 seconds later each time, when QueryAPIGate's copy of the schema expires.

So a governed key pays about 75 ms every minute, more on the very first call. Only the few requests that arrive while it happens wait, so it shows at p99.99, not at p99.9. The plain key has no table check or masking to redo, and no spikes.

The access log

`main`, the lookup, 32 callers, gunicorn's access log on and off (session A):

ACCESS LOG	REQ/S	P50	P99
on (the Docker image's default)	736 (735-739)	43.0 ms	54.4 ms
off	785 (782-792)	40.2 ms	51.7 ms

- **The access log costs 7%.**
- **QueryAPIGate also logs two lines of its own for each request,** at INFO, which can't be turned off. The profile below puts all logging at a fifth of the gateway's CPU.

A database across a network

netem on the database container delays everything PostgreSQL sends by 0.5 ms (`tc qdisc add dev eth0 root netem delay 0.5ms`; ping goes from 0.04 to 0.56 ms). That's about a same-zone cloud round trip. The lookup, session B:

	V0.41.0	MAIN	CACHED
One caller: req/s	133 (132-134)	162 (161-168)	270 (267-273)
One caller: p50	7.57 ms	6.24 ms	3.65 ms
32 callers: req/s	674 (670-753)	735 (734-741)	855 (852-874)

- **One caller pays for every round trip.** At 0.5 ms each, a call on `main` went from 1.4 to 6.2 ms.
  - Fewer store reads made `main` 21% faster than v0.41.0.
  - The cache made it 67% faster again, and cut single-call latency by 41%, to 3.65 ms.
- **With 32 callers, round trips overlap,** and the CPU is the limit again, so the builds come out in the same order as without a delay.
- **v0.41.0's range overlaps `main`'s here,** in both sessions. One of its five rounds ran at 753 requests a second, at 4.09 GHz against about 3.62 for the other four. Its median is 9% below `main`'s.
  - Both times, that was the block's first run, straight after the 10 s `/health` floor, on a CPU with turbo headroom to spend.
  - The rotation starts each part of the block with the same build, so v0.41.0 got that first slot twice. A harness fix would carry the rotation across the parts, or run one unrecorded warm-up first.
- **This is a simulation:** one-way delay, no packet loss or jitter. The next section uses it to count round trips. It isn't a cloud measurement.

Mixed reads and writes, and hot rows

`main` (session B):

WORKLOAD	REQ/S	P50	P99
Reads (24 callers) and writes (8 callers) at once	548 (546–549) + 181 (181–182)	43.3 / 43.6 ms	54.4 / 54.9 ms
Writes on 10 million accounts (32 callers)	734 (733–736)	43.1 ms	54.4 ms
Writes on 99 hot accounts (32 callers)	738 (735–740)	42.9 ms	53.9 ms

- **Mixing reads and writes changes nothing:** they share the process's capacity in proportion, 729 a second in total.
- **Writes on 99 hot rows cost the same as on 10 million.** One worker sends at most 8 at once, so PostgreSQL's row locks never queue long enough to show. With many workers, or a slow transaction, they would.

Where the time goes

Round trips per call

Counted. `log_statement = 'all'` was turned on for each database, then 100 calls were made after a warm-up:

STATEMENTS PER CALL	V0.41.0	MAIN	CACHED
Store, plain key or writer	7: 5 reads, 2 inserts	3: the key, the query, the connection	0.1: history, in batches
Store, governed key	8	4	1: the masking hash key
Data database, any call	4: `BEGIN READ ONLY`, `SET LOCAL statement_timeout`, the query, `ROLLBACK`	4	4
Round trips a call waits for (the history inserts happen in the background)	**9**	**7**	**4**

Measured. Single-call p50 at 0, 0.5, 1 and 2 ms of netem delay, 3 rounds per point (session B). The slope of latency against the delay is the number of round trips a call waits for. psycopg2 on its own, sending a known 1 or 4 statements, calibrates the method:

	+0 MS	+0.5 MS	+1 MS	+2 MS	SLOPE	ROUND TRIPS (SLOPE ÷ 1.15)
psycopg2, 1 statement	0.03	0.63	1.19	2.33	1.15	1
psycopg2, the 4 statements a query sends	0.06	2.44	4.68	9.36	4.60	4.0
v0.41.0	1.42	7.60	12.47	22.66	10.46	9.1
main	1.43	6.24	10.01	18.75	8.58	7.5
Cached	1.05	3.70	5.84	10.59	4.72	4.1

- **Under netem, one round trip costs 1.15 ms per ms of delay,** not 1. Bare psycopg2 shows the same factor, at 1 statement and at 4. It is netem's per-packet overhead, not the gateway.
- **Divided by it, the measured round trips match the counted ones:** 9.1 against 9, 7.5 against 7, 4.1 against 4. The statement log missed nothing. (A v2 review had asked whether it did: v2's raw 0.5 ms figures implied 1-3 round trips more than counted.)
- **History and events** are written in batches of up to 500 by a background thread, about once a second, so a call doesn't wait for them.
- **The governed key reads the masking hash key on every call,** even with the cache on.
- **A query takes four round trips to the data database:** a transaction, its time limit, the statement, then the rollback (or commit, for a write).

The CPU

py-spy at 200 samples a second across the worker's threads, during one 30 s run of the lookup at 32 callers (session A). Each sample is counted under the innermost frame that falls in one of these categories:

WHERE THE SAMPLES FALL	MAIN	CACHED
Store layer: queries, decoding rows, checks	28.2%	13.4%
Logging: the access log and QueryAPIGate's own lines	18.4%	20.4%
gunicorn, sockets, HTTP	16.8%	21.0%
Data query: the driver and the pool	12.3%	15.4%
Flask and Werkzeug	9.9%	11.9%
QueryAPIGate's request handling	4.6%	5.6%
History writer thread	3.0%	3.7%
Authentication, limits, quotas, metrics	2.5%	2.8%
SQL checks, parameters, masking	1.7%	1.9%
JSON	1.3%	2.6%
Other	1.3%	1.3%

The busiest single frames say more:

- `logging` **lock acquisition: 8-10%.** Eight threads write log lines through one handler, and wait on its lock. That's the same contention that makes more threads slower.
- `store.refuse_store_access` **: 7-10%.** A safety check that a connection doesn't point at QueryAPIGate's own SQLite file. It resolves file paths (`os.path.realpath` , several system calls) on every call - even for a PostgreSQL connection, which has no file to point at. With the cache on, it's the largest single cost in the store layer.
- **The store's** `execute` **: 17% in main, gone with the cache.** That's the three reads.
- **gunicorn's per-request worker heartbeat (** `workertmp.notify` **): 3%.**

Sizing

A worker has two limits, and it reaches whichever comes first:

- **CPU.** About 550 calls a second per worker, or 650 with the metadata cache, for fast OLTP calls at p99 of about 4 ms. That's 75% of the capacity above. Every millisecond of Python a call costs beyond this page's lookup lowers it.
- **Waiting.** A worker holds at most as many calls at once as it has threads. A call waits for its query, plus its round trips (7 on `main` , 4 cached), each taking the network's round-trip time. So a worker serves at most: threads ÷ (query time + round trips × round-trip time) calls a second.

Workers needed = the larger of (calls a second ÷ the CPU figure) and (calls a second ÷ the waiting figure), plus one to lose without falling over.

Two examples, at a 0.5 ms network round trip:

CALLS	CPU FIGURE	WAITING FIGURE, 8 THREADS	WHAT LIMITS A WORKER
A 0.03 ms lookup, as on this page	550 a second	8 ÷ (0.03 + 7 × 0.5 ms) = about 2,270 a second	CPU: fewer threads would do (see Threads)
A 20 ms report query	550 a second, or fewer	8 ÷ (20 + 3.5 ms) = about 340 a second	Waiting: give it more threads, as long as CPU allows

These are planning figures from one machine and one query shape. Measure your own queries at 75% of what you plan to serve before you commit to a number.

What to do with this

For running QueryAPIGate:

- **Size with the rule above,** then measure your own queries, which will usually cost more than these.
- **Turn on** `QUERYAPIGATE_METADATA_CACHE_TTL` if a few seconds' delay is acceptable for a change made on another instance to reach this one. A worker then serves 16-23% more, PostgreSQL does half the work, and across a network a single call takes 41% less time. The trade-off is explained in PERFORMANCE.md.
- **Match threads to your queries:**
  - For fast queries, `--threads 2` beats the default 8 by 15% here.
  - Keep 8 or more if calls wait on slow queries, exports or `/events` .
- **Turn off gunicorn's access log** if a proxy in front already logs requests: +7%.
- **Scale with processes on their own cores.** A hyperthread gave about half a core here.

For QueryAPIGate itself - findings worth an issue each, in the order I'd take them:

1. **Fewer round trips per query.** Set `default_transaction_read_only` and `statement_timeout` once per pooled session, or send `BEGIN` and `SET LOCAL` in one round trip with the query.
   - That takes 4 round trips down to 1 or 2.
   - At a 0.5 ms round trip, saving 3 of them takes about 1.7 ms off every call: 6.2 to about 4.5 ms on `main`, and 3.7 to about 2 ms cached.
2. **Skip `refuse_store_access`'s path checks for server databases.** Only SQLite, DuckDB and H2 open a file. That would save 7–10% of CPU on every PostgreSQL, MySQL, ClickHouse and Trino call.
3. **Cache the masking hash key.** It's read on every governed call, and almost never changes.
4. **Make logging cheaper:** a setting to skip the per-request INFO lines, or a queue handler so threads don't wait on one lock. That's up to a fifth of the CPU.
5. **Refresh the schema and masking trace in the background,** before they expire. Then no governed call waits the extra 75 ms each minute, or the 147 ms on its first call.

What changed from v2

A review of v2 rated it 4.1 out of 5. What it raised, and what v3 did about it:

RAISED	DONE
"Halved single-call latency": 6.24 → 3.69 ms is -41%, not half, and another section says 40%	-41% throughout
"No caller waits 60–140 ms every minute": the recurring spikes are about 75 ms, and 147 ms is the first call	"About 75 ms every minute, more on the first call", with why the first spike is at t=1
v1's 2× gap credited to power alone, though v2 also pinned cores and removed Docker's proxy	All three are credited (below)
"Every number is a median with its range", though some tables showed medians only	Ranges in every comparison table; the single long runs are marked
3 rounds give a 10% chance of a false "clear"	Every 3-round block was rerun with 5 (under 1%), and the rule's odds are stated in the method
The threads result cleared by 3 req/s, in a 3-round block	With 5 rounds, every threads range is clear: 2 threads are best, and 8 trail 1 by 7%
Counted round trips didn't explain the network result	A calibrated slope test: they match once netem's 1.15× per-trip overhead is accounted for
Closed-loop "capacity" was below what was served at 110%	Capacity is now measured in the same block; the 2–4% gap is stated
The clock range was recorded but never used	±1% within cells; turbo spikes explain the outlier rounds
Load generator CPU not recorded	Recorded per run: at most 0.79 of its 8 cores
The short answer named 4 round trips but not the 3 to the store	7 on `main`, 4 cached
Unfinished requests at 110% might be read as part of "every request returned 200"	Stated: requests still queued when a run ended are counted neither way
Run time stated two ways	One figure, under Run it yourself

Carried over from v2's corrections to v1:

- **The store transactions per call.** v1 said about 5.5 before its changes and 2.6 after. Counted exactly, it's 7 statements before (5 reads and 2 inserts) and 3 after. PostgreSQL holds back an idle session's statistics for up to 10 seconds, and v1 read them about 2 seconds after its run ended.
- **v1's PostgreSQL throughput was about half of this page's:** 368 requests a second for `main` against 743 here. The code was the same. Three things changed at once: the laptop went from battery and a power-saving profile to AC and `performance`; every part got its own cores; and the gateway stopped reaching PostgreSQL through Docker's userland proxy. No run changed one at a time, so this page doesn't say how much each was worth. The power state is likely most of it, because `/health`, which doesn't touch PostgreSQL, doubled as well.

Limits

- **One machine.** The gateway, the database and the load generator share it, on separate cores but one memory bus and one cooling system. Four physical cores for the gateway limit what scale-out can show.
- **No TLS, no load balancer or proxy, and no Redis,** so there are no shared rate limits. Each of them would add to the cost of a call.
- **The network is simulated,** one way, without jitter or loss.
- **Uniform random keys.** Real traffic is skewed. Hot rows were tested for writes, but not as a skewed read pattern, which would matter to a cache in front of the database.
- **Small results only:** one row, or a page of 100. Large results, `?stream=true`, the MCP server and the other databases weren't measured.
- **p99.99 and the slowest requests** come only from the 150 s runs. Elsewhere, 30 s runs give about 20,000 samples, enough for p99.9.
- **py-spy counts samples of threads it considers active,** and a thread waiting for the interpreter lock can count. So the shares say where the time is spent, not exactly how much each part would save.
- **The first run after a floor gets a turbo burst.** A block's rotation starts its first round with the same build in each part of the block, so that build gets the burst more often - see the network section.
- **No soak or failure tests.** Hours at 75% load, which would show memory, pool and history-writer behaviour over time, weren't run. Neither was losing PostgreSQL or a worker mid-run.

Run it yourself

You need Docker, `taskset`, and QueryAPIGate's server, PostgreSQL and masking dependencies in the Python that runs the script. py-spy is optional, for the profile:

```shell
pip install -e ".[server,postgres,flow]" py-spy
python benchmarks/oltp.py --out /tmp/qag-oltp setup all teardown     # about 3 hours 45 minutes
python benchmarks/oltp_report.py /tmp/qag-oltp/results.jsonl         # writes results.md beside it
```

- **Cores and refs:** `--gateway-cpus`, `--db-cpus` and `--load-cpus` take `taskset`-style lists; check your own machine's layout with `lscpu -e`. `--base` and `--head` choose the two git refs (v0.41.0 and `origin/main` by default).
- **Fewer blocks:** name them instead of `all`, e.g. `setup env b1 b2 teardown` - each block's docstring says what it measures. `--smoke` runs every block for 2 s, to check the setup in about 10 minutes.
- **Plug in first.** Check `powerprofilesctl get` and the AC state; the `/health` floors will show it if you forgot.
- **What it leaves behind:** `teardown` removes the database container and its data volume, about 2 GB.